

Dokumen Bisnis : Veritas Shield

1 Model Bisnis

Veritas Shield merupakan sebuah layanan berbasis aplikasi (SaaS) yang menyediakan API peringatan bahasa kasar yang cepat dan mudah diintegrasikan, memungkinkan pengguna menyaring konten teks dengan daftar aturan yang dapat disesuaikan, ideal untuk kebutuhan moderasi dasar dan cepat.

2 Layanan Utama

Layanan utama yang ditawarkan oleh Veritas Shield melalui API adalah sebagai berikut:

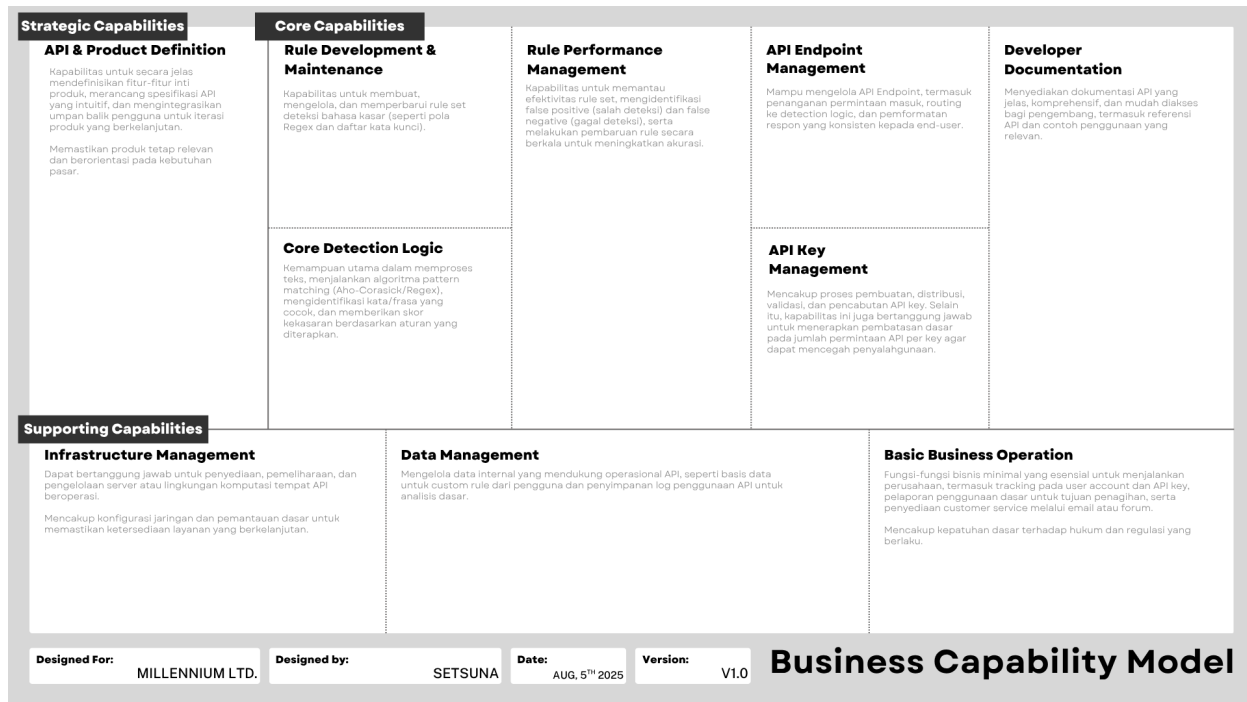
- a. **Deteksi Bahasa Kasar:** Menerima teks masukan dan mengembalikan indikasi apakah teks tersebut mengandung bahasa kasar atau tidak berdasarkan aturan pattern matching.
- b. **Identifikasi Kata/Frasa Spesifik:** Mengembalikan daftar kata atau frasa kasar yang terdeteksi.
- c. **Dukungan Multi-Bahasa (via Pengaturan):** Kemampuan untuk mendeteksi bahasa kasar dalam berbagai bahasa jika aturan pattern matching untuk bahasa tersebut disediakan.
- d. **Konfigurasi Kustom Pengguna:** Memungkinkan pengguna untuk menambahkan daftar kata-kata kasar mereka atau mengecualikan kata-kata tertentu melalui API.

3 Strategi Bisnis

Strategi bisnis yang dimiliki Veritas Shield berfokus pada layanan freemium sederhana untuk menarik pengguna dengan tingkat gratis, sekaligus menawarkan opsi berlangganan dasar yang jelas dan terjangkau, serta membuka peluang donasi sebagai bentuk dukungan. Target utama dari Veritas Shield adalah pengembang aplikasi.

4 Business Capability Model

Berikut adalah diagram *Business Capability Model* dari Veritas Shield



Gambar 4.1. Diagram *Business Capability Model*

4.1 Analisis Teknologi yang Dibutuhkan

Pengembangan dan operasional layanan (SaaS) Veritas Shield oleh Millenium Ltd. akan memerlukan kombinasi teknologi di berbagai lapisan. Berikut adalah analisis teknologi utama yang dibutuhkan:

4.1.1 Teknologi Pattern Matching & Rule Management

- Programming Language:** Python dipilih karena kemudahan penggunaan, ekosistem yang kaya, dan dukungan bawaan untuk *regex*.
- Regex Library:** Re (modul bawaan Python).
- Aho-Corasick Library:** Dapat menggunakan *library* Python yang sudah ada (*pyahocorasick*, *flashtext*).
- Struktur Data untuk Rule:**
 - Daftar kata kasar inti dan *user customized* disimpan di dalam satu database.
- Rule Management System:** Manual atau melalui API untuk *user customized list*.

4.1.2 Teknologi Backend & API

- Programming Language:** Python.
- Web Framework:**
 - FastAPI:** Merupakan pilihan utama karena ringan, cepat, modern, dan secara otomatis menghasilkan dokumentasi API (OpenAPI/Swagger UI) yang akan sangat menghemat waktu saat membuat dokumentasi.
 - Flask:** Alternatif yang lebih minimalis.
- Authentication & Authorization (OAuth2):**

- i. **OAuth2 Library:** Menggunakan *library* Python untuk implementasi OAuth2 (Authlib atau Flask-OAuthlib jika menggunakan Flask) untuk membantu dan mengelola alur validasi.
- ii. **JSON Web Tokens (JWT):** Token akses akan diimplementasikan sebagai JWT untuk validasi yang efisien tanpa perlu *lookup* database pada setiap permintaan API menggunakan *library* PyJWT.
- d. **Rate Limiting:** Dapat diimplementasikan secara sederhana di dalam kode aplikasi menggunakan *in-memory cache* (*library* cachetools, functools.lru_cache di Python) atau dengan Redis sebagai alternatif dengan kompleksitas minimal.

4.1.3 Teknologi Database

- a. **Database Utama:**
 - i. **PostgreSQL:** *Relational* database yang kuat untuk menyimpan API keys, daftar kata *user customized*, dan *usage log*.
 - ii. **SQLite:** Alternatif yang dapat digunakan apabila ingin database yang *serverless*, mudah dipahami.
- b. **ORM/Database Library:** SQLAlchemy (Python *library*) atau *library* bawaan *framework* (ORM Flask/FastAPI) untuk berinteraksi dengan database.

4.1.4 Cloud Infrastructure (Hosting)

- a. **Penyedia Cloud:**
 - i. **Render (Free Tier):** Menawarkan *free tier* untuk layanan web dan database.
 - ii. **Railway (Free Tier):** Pilihan lain dengan *free tier* yang murah.
 - iii. **DigitalOcean Droplet:** Menawarkan kontrol lebih besar dengan biaya minimal (sekitar \$4-6/bulan).
 - iv. **Local Deployment (Ngrok):** Untuk demo atau pengujian awal, bisa menjalankan API secara lokal dan mengesposnya ke internet menggunakan Ngrok, meskipun ini bukan solusi untuk produksi.
- b. **Layanan Komputasi:** Satu instance VM kecil atau *container* tunggal.
- c. **CI/CD:** Manual deployment atau GitHub Actions dasar untuk otomatisasi *build* dan *deploy* yang sangat sederhana.

4.1.5 Teknologi DevOps & Pemantauan

- a. **Deployment:** Manual git push ke *repository* dan langsung di handle Render/Railway, atau SSH ke VM untuk git pull.
- b. **Pemantauan & Logging:** Log aplikasi ke *console/file*, dan pantau melalui *dashboard* cloud service.

4.1.6 Teknologi Developer Portal

- a. **Dokumentasi API Otomatis:** FastAPI akan menghasilkan endpoint */docs* (Swagger UI) secara otomatis. Merupakan cara termudah dan tercepat untuk menyediakan dokumentasi.
- b. **Static Webpage:** Halaman HTML/Markdown sederhana di GitHub Pages atau Netlify untuk informasi dasar dan pendaftaran aplikasi klien.

5 Functional Requirements

ID	Kebutuhan	Penjelasan
F01	Deteksi bahasa kasar	Sistem harus mampu menerima teks sebagai masukan dan mengembalikan indikasi apakah teks tersebut mengandung bahasa kasar atau tidak berdasarkan aturan yang telah ditentukan.
F02	Peringatan bahasa kasar	Sistem harus mampu mengembalikan peringatan bahasa kasar apabila indikasi menyatakan bahwa teks mengandung bahasa kasar berdasarkan aturan yang telah ditentukan.
F03	Identifikasi kata atau frasa	Sistem harus mampu mengidentifikasi dan mengembalikan daftar kata atau frasa spesifik yang terdeteksi sebagai bahasa kasar.
F04	Dukungan multi-bahasa	Sistem harus mendukung deteksi bahasa kasar untuk setidaknya Bahasa Indonesia dan Bahasa Inggris jika aturan <i>pattern matching</i> yang relevan tersedia.
F05	Konfigurasi pengguna	Pengguna harus dapat menambahkan kata-kata ke <i>blacklist</i> mereka atau <i>whitelist</i> (pengecualian) melalui API.
F06	Autentikasi OAuth2	Setiap permintaan API harus diautentikasi menggunakan <i>access token</i> OAuth2 yang valid.
F07	<i>Rate Limiting</i>	Sistem harus menerapkan pembatasan jumlah permintaan API per unit waktu untuk setiap API Key.
F08	<i>Logging Request</i> Dasar	Sistem harus mencatat setiap permintaan API untuk tujuan pemantauan dan penagihan dasar.
F09	Pendaftaran Aplikasi Klien	Sistem harus menyediakan mekanisme sederhana bagi pengguna untuk mendaftarkan aplikasi mereka dan mendapatkan Client ID serta Client Secret.
F10	Penerbitan <i>Access Token</i>	Sistem harus mampu menerbitkan <i>access token</i> OAuth2 (misalnya, melalui alur Client Credentials Grant untuk aplikasi server-to-server) setelah validasi Client ID dan Client Secret.
F11	Validasi <i>Access Token</i>	Sistem harus mampu memvalidasi <i>access token</i> yang diterima pada setiap permintaan API untuk memastikan keaslian dan masa berlakunya.

Tabel 5.1. Tabel *Functional Requirements*

6 Use Case

6.1 Identifikasi Actor

Kode Actor	Nama	Keterangan
A01	Pengembang	Menggunakan API untuk memeriksa adanya kata kasar dalam teks, memberikan peringatan, dan melakukan pengaturan.

A02	Calon Pengguna (Pengembang)	Melakukan registrasi agar dapat menggunakan API
-----	-----------------------------	---

Tabel 6.1. Tabel Identifikasi Actor

6.2 Identifikasi Use Case

No	Use Case	Keterangan
UC01	Mendeteksi Bahasa Kasar dalam Teks	Sistem dapat mendeteksi bahasa kasar dalam teks dan memberikan peringatan.
UC02	Mengelola Daftar Kata	Pengembang dapat menambahkan atau mengurangi daftar kata pada <i>blacklist</i> atau <i>whitelist</i> mereka.
UC03	Mendaftarkan Aplikasi Klien & Mendapatkan Client Credentials	Calon pengguna (pengembang) dapat melakukan registrasi aplikasi klien.
UC04	Mendapatkan <i>Access Token</i> OAuth2 (Client Credentials Grant)	Aplikasi pengembang dapat mendapatkan <i>access token</i> untuk otorisasi <i>request</i> ke API.

Tabel 6.2. Tabel Identifikasi Use Case

6.3 Skenario Use Case

6.3.1 Skenario Use Case 01

Nama Use Case: Mendeteksi Bahasa Kasar dalam Teks

Skenario: 01

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Melakukan HTTP POST <i>request</i> ke <i>endpoint</i> /detect dengan teks sebagai payload dan <i>access token</i> yang valid sebagai header	
	2. Memvalidasi <i>access token</i> dan melakukan pembatasan tingkat, memproses teks dan mengembalikan JSON <i>response</i> yang menunjukkan teks tersebut kasar, dan kata/frasa yang terdeteksi
<i>Skenario Alternatif</i>	
1. Melakukan HTTP POST <i>request</i> ke <i>endpoint</i> /detect dengan teks sebagai payload	
	2. Mengembalikan JSON <i>response Unauthorized</i>

Tabel 6.4.1. Tabel Skenario Use Case 01

6.3.2 Skenario Use Case 02

Nama Use Case: Mengelola Daftar Kata

Skenario: 02

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Melakukan HTTP POST/PUT/DELETE <i>request</i> ke <i>endpoint</i> /custom-words dengan kata/frasa	

yang ingin dihapus/ditambahkan dan <i>access token</i> yang valid sebagai header	
	2. Memvalidasi <i>access token</i> , melakukan pembaruan pada database dan mengembalikan JSON <i>response</i> OK
<i>Skenario Alternatif</i>	
1. Melakukan HTTP POST/PUT/DELETE <i>request</i> ke <i>endpoint</i> /custom-words tanpa token yang valid	
	2. Mengembalikan JSON <i>response Unauthorized</i>

Tabel 6.4.2. Tabel Skenario Use Case 02

6.3.3 Skenario Use Case 03

Nama Use Case: Mendaftarkan Aplikasi Klien & Mendapatkan Client Credentials

Skenario: 03

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Melakukan registrasi di website/portal layanan dengan mengisi nama aplikasi, redirect URI (jika ada), dan email	
	2. Menyimpan informasi ke dalam database dan menampilkan Client ID dan Secret

Tabel 6.4.3. Tabel Skenario Use Case 03

6.3.4 Skenario Use Case 04

Nama Use Case: Mendapatkan Access Token OAuth2 (Client Credentials Grant)

Skenario: 04

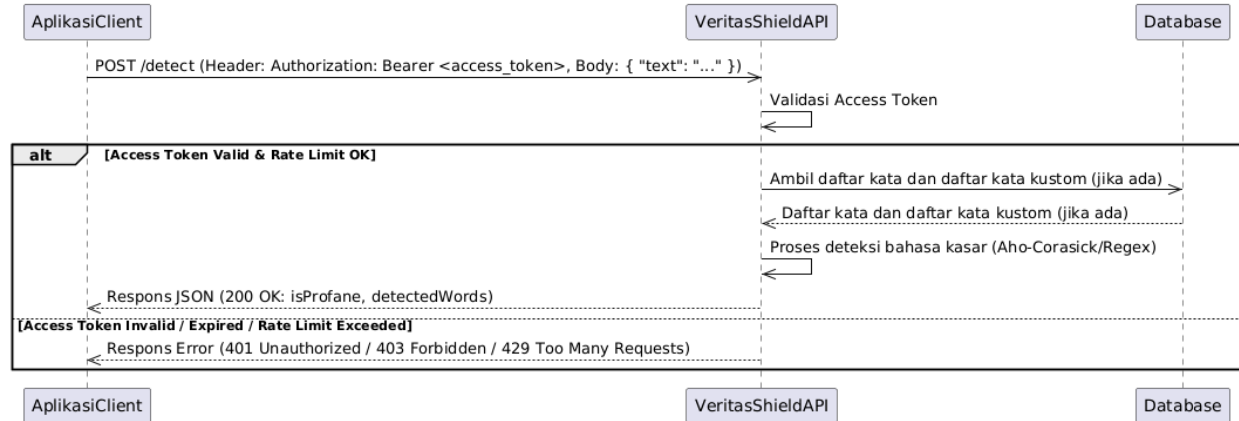
<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Melakukan HTTP POST <i>request</i> ke <i>endpoint</i> /oauth/token (Authentication Server) dengan Client ID dan Secret (di header Authorization: Basic atau body form-urlencoded) dan grant_type=client_credentials	
	2. Memvalidasi Client ID dan Secret, kemudian menerbitkan <i>access token</i> dengan masa berlaku tertentu, dan mengembalikan JSON <i>response</i> berisi <i>access token</i> dan masa berlaku

Tabel 6.4.4. Tabel Skenario Use Case 04

7 Sequence Diagram

7.1 Use Case Mendeteksi Bahasa Kasar dalam Teks

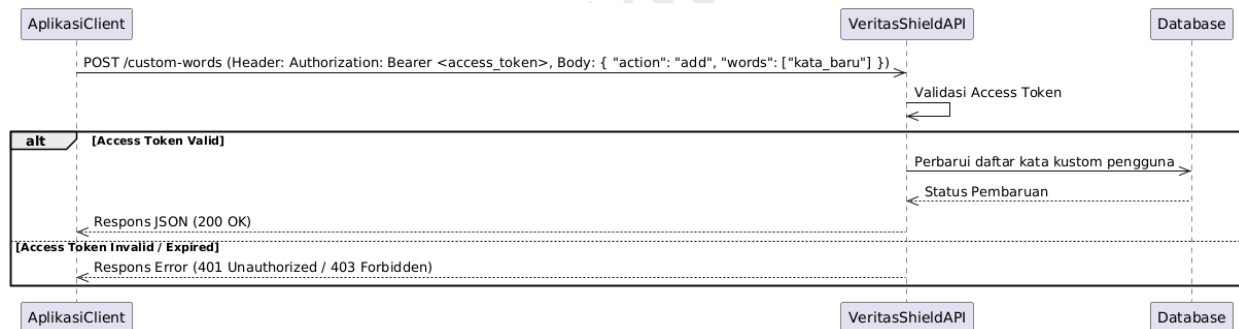
7.1.1 Sequence Diagram



Gambar 7.1.1. Sequence Diagram Use Case 01

7.2 Use Case Mengelola Daftar Kata

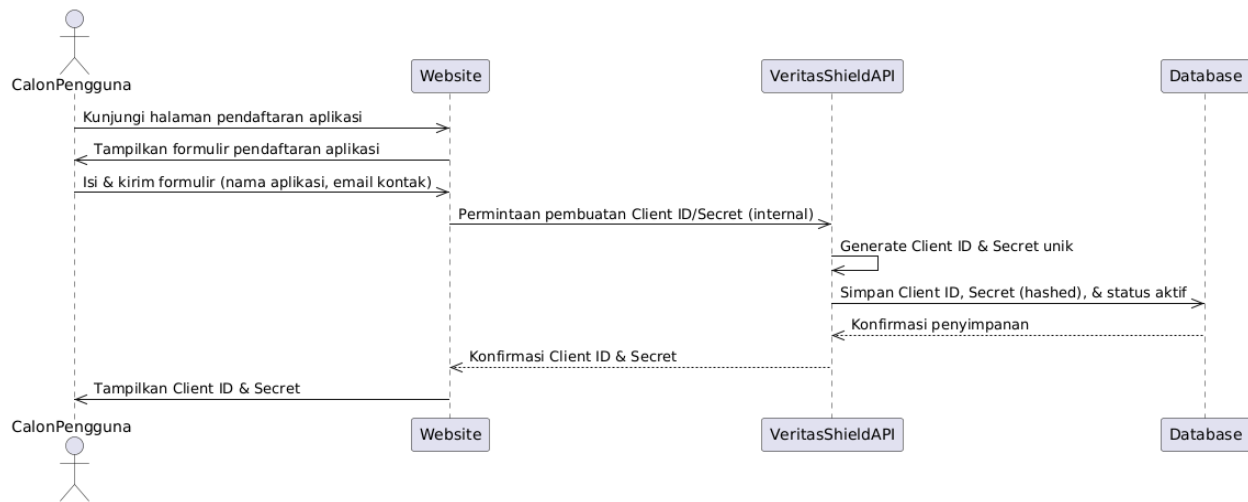
7.2.1 Sequence Diagram



Gambar 7.2.1. Sequence Diagram Use Case 02

7.3 Use Case Mendaftarkan Aplikasi Klien & Mendapatkan Client Credentials

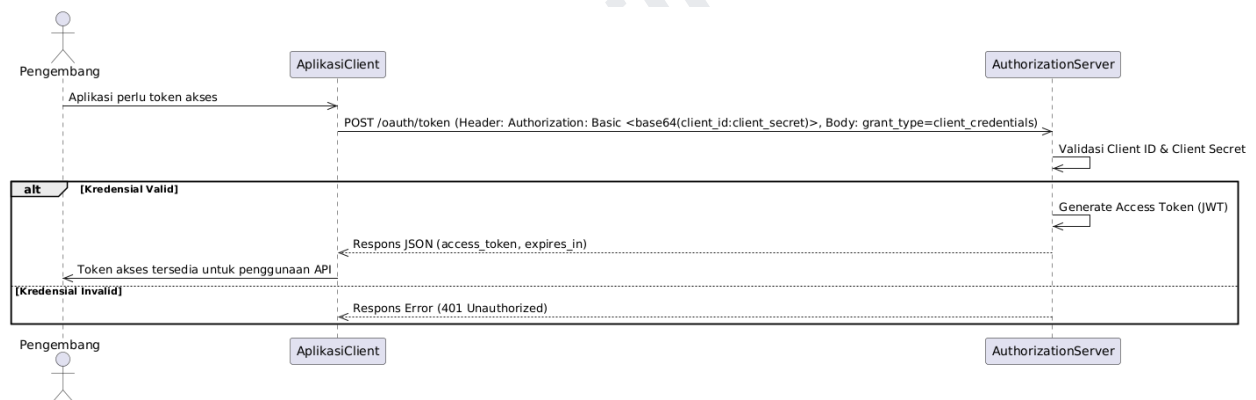
7.3.1 Sequence Diagram



Gambar 7.3.1. Sequence Diagram Use Case 03

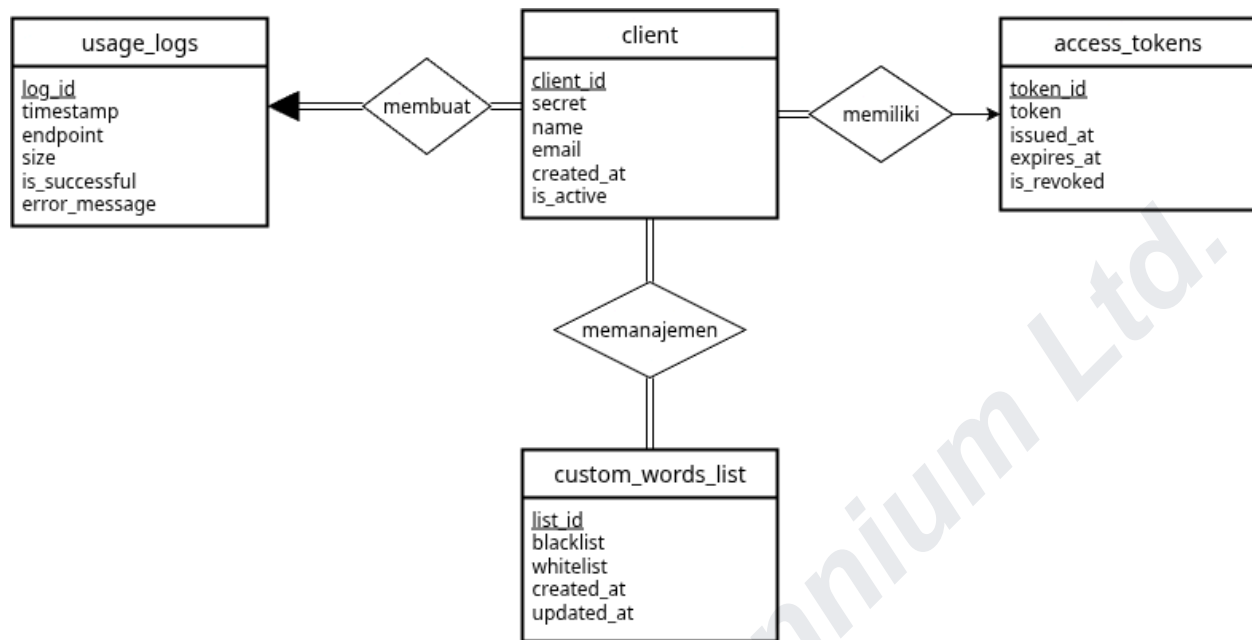
7.4 Use Case Mendapatkan Access Token OAuth2 (Client Credentials Grant)

7.4.1 Sequence Diagram



Gambar 7.4.1. Sequence Diagram Use Case 04

8 Entity-Relationship Diagram



Gambar 8.1. E-R Diagram Veritas Shield

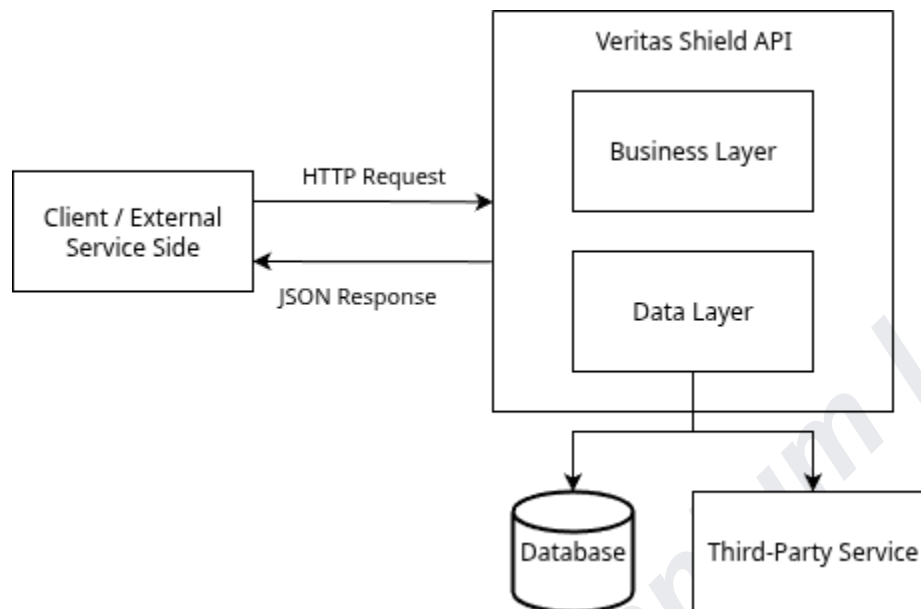
Entitas **client** menyimpan informasi dasar pengguna dengan atribut **client_id**, **name**, **email**, **secret**, **created_at**, dan **is_active** yang terhubung dengan **access_tokens** (one-to-many) melalui relasi **memiliki**, **custom_words_list** (one-to-one) melalui relasi **memanajemen**, dan **usage_logs** (one-to-many) melalui relasi **membuat**.

Entitas **access_tokens** menyimpan informasi mengenai token yang digunakan untuk mengakses layanan API dengan atribut **token_id**, **token**, **issued_at**, **expires_at**, **is_revoked**.

Entitas **custom_words_list** menyimpan informasi mengenai daftar kata kustom milik pengguna dengan atribut **list_id**, **blacklist**, **whitelist**, **created_at**, dan **updated_at**.

Entitas **usage_logs** menyimpan informasi mengenai log pemakaian dari API dengan atribut **log_id**, **timestamp**, **endpoint**, **size**, **is_successful**, dan **error_message**.

9 Application Architecture Diagram



Gambar 9.1. Veritas Shield Application Architecture Diagram

- a. **Client / External Service Side:** *End-user* yang mengintegrasikan API.
- b. **Veritas Shield API:** Layanan tunggal yang terdiri dari dua layer.
 - i. **Business Layer:** Menjalankan logika bisnis seperti menerima *request*, memvalidasi *access token* dan melakukan *rate limiting*, memproses teks dengan *pattern matching / regex*, dan berinteraksi dengan Data Layer.
 - ii. **Data Layer:** Berinteraksi dengan Business Layer untuk memberikan data dari Database dan Third-Party Service.
- c. **Database:** Menyimpan data pengguna, *access token*, *usage logs*, dan daftar kata.
- d. **Third-Party Service:** Layanan pihak ketiga yang digunakan untuk mengurus pembayaran.